

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ЛИПЕЦКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Институт компьютерных наук

Кафедра автоматизированных систем управления

Индивидуальное домашнее задание
по дисциплине
«Управление ИТ-проектами»

Студент М-РИТ-25-1 _____ Станиславчук С.М.

Руководитель

К.т.н., доцент _____ Ведищев В.В.

Липецк, 2026

Содержание

1. Разработка прототипа ИТ-проекта 3
1.1 Используемые средства для разработки прототипа 3
1.1.1 Язык программирования 3
1.1.2 Сетевое взаимодействие 4
1.1.3 Графический интерфейс 4
1.1.4 Голосовой движок 5
1.1.5 Обработка изображений 6
1.1.6 Виртуальное окружение и управление зависимостями 6
1.1.7 Контроль версий 7
1.2 Функционал прототипа 7
1.2.1 Текстовые сообщения 7
1.2.2 Голосовой канал 8
1.2.3 Передача файлов (изображений на проектор) 9
1.3 Запуск прототипа 10
1.3.1 Шаг 1. Сервер 11
1.3.2 Шаг 2. Клиент 11
1.3.3 Шаг 3. Клиент проектора (опционально) 13
1.3.4 Шаг 4. Подключение к нужной комнате 13

1. Разработка прототипа ИТ-проекта

1.1. Используемые средства для разработки прототипа

Разработка прототипа программного комплекса «CorpCom» выполнена с использованием языка Python 3.13 и стека библиотек, обеспечивающих сетевое взаимодействие в реальном времени, графический интерфейс и потоковую обработку звука. Выбор инструментов обоснован требованиями к прототипу: работа в локальной сети предприятия без внешних облачных сервисов, минимальные задержки голосовой связи, кроссплатформенность клиентской части.

1.1.1. Язык программирования

В качестве основного языка разработки выбран Python (версия 3.13). Решение мотивировано следующими факторами:

- Скорость прототипирования. Python позволяет реализовать WebSocket-сервер, клиентский GUI и голосовой движок в считанные часы благодаря лаконичному синтаксису и богатой экосистеме библиотек.
- Асинхронная модель. Встроенный модуль `asyncio` обеспечивает конкурентную обработку десятков WebSocket-соединений на одном потоке без накладных расходов на многопоточную синхронизацию.
- Кроссплатформенность. Код выполняется на любой ОС с установленным интерпретатором Python (Linux, macOS, Windows), что упрощает развёртывание прототипа в разнородной корпоративной среде.

1.1.2. Сетевое взаимодействие

Для реализации транспортного уровня протокола выбрана библиотека `websockets` (версия ≥ 12.0). Библиотека предоставляет:

- асинхронный сервер (`websockets.serve`) с встроенной поддержкой `ping/pong` для обнаружения разрывов соединения;
- асинхронный клиент (`websockets.connect`) с контекстным менеджером, автоматически закрывающим соединение;
- потоковый приём сообщений через `async for` по сокету;
- настраиваемый лимит размера входящих сообщений (`max_size`) — критично для передачи изображений.

Протокол обмена — JSON поверх WebSocket (RFC 6455). Все сообщения кодируются в UTF-8 и передаются как текстовые фреймы. Бинарные данные (изображения, PCM-аудио) кодируются в Base64 и вкладываются в JSON-поле `data`. Такой подход упрощает отладку: любое сообщение можно прочитать в консоли браузера или Wireshark без десериализации бинарного потока.

1.1.3. Графический интерфейс

Клиентский GUI реализован на библиотеке `tkinter` (стандартный модуль Python, обвязка над Tcl/Tk). Выбор обоснован:

- отсутствием внешних зависимостей — tkinter входит в стандартную поставку Python;

- достаточной функциональностью для прототипа: текстовые поля, кнопки, скроллируемый чат, диалоги выбора файлов, превью изображений;
- событийной моделью, совместимой с асинхронным сетевым кодом через очередь `queue.Queue` и периодический опрос `root.after()`.

GUI разделён на смысловые панели: «Подключение» (адрес, порт, имя, комната, токен), «Голос в реальном времени» (микрофон, PTT, приём), «Чат» (история сообщений, поле ввода) и «Медиа» (отправка изображения на проектор).

Для клиента проектора также используется tkinter с полноэкранным режимом (`attributes(fullscreen")`) и автоматическим масштабированием изображения под размер канваса.

1.1.4. Голосовой движок

Потоковая обработка звука реализована на библиотеках `sounddevice` (захват и воспроизведение PCM через PortAudio) и `numpy` (работа с массивами семплов). Параметры аудио:

- частота дискретизации: 16 000 Гц;
- разрядность: 16 бит;
- канальность: моно;
- размер блока: 320 семплов (20 мс).

Выбор 16 кГц моно обусловлен компромиссом между качеством речи и пропускной способностью сети: каждый 20-миллисекундный блок

занимает 640 байт (без учёта Base64-кодирования и JSON-обёртки), что даёт поток около 32 КБ/с — приемлемо даже для Wi-Fi сетей предприятия.

Захват звука реализован через `sd.InputStream` с callback-функцией, которая передаёт блоки байт в асинхронную очередь `asyncio.Queue`. Воспроизведение — через `sd.OutputStream` с непрерывным callback-чтением из потокобезопасной очереди `queue.Queue`. При переполнении очереди старые блоки отбрасываются — это предотвращает накопление задержки.

1.1.5. Обработка изображений

Для работы с изображениями используется библиотека `Pillow` (PIL, версия ≥ 10.0). В клиенте Pillow применяется для:

- чтения файлов изображений с диска в бинарный буфер (`Image.open`);
- создания уменьшенного превью (320×180) для отправителя;
- масштабирования полученного изображения под размер канваса проектора (`Image.thumbnail` с фильтром `LANCZOS`).

1.1.6. Виртуальное окружение и управление зависимостями

Зависимости проекта зафиксированы в файле `requirements.txt` и устанавливаются в изолированное виртуальное окружение `.venv/`. Скрипт `setup.sh` автоматизирует создание окружения и установку пакетов, а скрипт `corpcom_exec.sh` обеспечивает запуск любого модуля с корректным `PYTHONPATH` и `PATH`, обходя проблему подмены `sys.executable` в IDE Cursor.

1.1.7. Контроль версий

Исходный код размещён в системе контроля версий Git, что позволяет отслеживать изменения на всех этапах разработки прототипа и возвращаться к предыдущим состояниям при необходимости.

1.2. Функционал прототипа

Прототип ПК «CorpCom» реализует три ключевые функции, обеспечивающие полноценный обмен информацией внутри рабочей группы: текстовые сообщения, голосовой канал и передачу файлов (изображений) на общий экран.

1.2.1. Текстовые сообщения

Обмен текстовыми сообщениями реализован по модели «комната» (chat room). Пользователь подключается к именованной комнате и получает сообщения от всех остальных участников этой комнаты в реальном времени.

Протокол. Текстовое сообщение передаётся как JSON-объект типа `MSG_TEXT` с полями: `room` (комната), `sender` (отправитель), `content` (текст). Сервер при получении текстового сообщения выполняет:

1. обрезку содержимого до 8000 символов (`content[:8000]`);
2. проверку на пустое сообщение (пустые игнорируются);
3. широковещательную рассылку всем пирам комнаты с ролью `user`, исключая отправителя.

Интерфейс. В клиенте история чата отображается в виджете `ScrolledText`, новые сообщения автоматически прокручиваются в конец.

Сообщения от других участников маркируются префиксом `[имя]`, собственные — префиксом `[я]`. Отправка производится по клавише `Enter` или кнопке «Отправить».

Системные уведомления. Сервер автоматически генерирует системные сообщения (`MSG_SYS`) при подключении и отключении участников, а также рассылает актуальный список участников комнаты (`MSG_CLIENT_LIST`) после каждого изменения состава.

1.2.2. Голосовой канал

Голосовая связь реализована как потоковая передача PCM-аудио поверх WebSocket в режиме реального времени. В отличие от традиционной VoIP-телефонии, голосовой канал CorpCom не требует установки выделенного медиа-соединения (SIP/RTP) — аудиоданные передаются через тот же WebSocket-канал, что и текстовые сообщения, что упрощает архитектуру и обходит проблемы с NAT/брандмауэрами.

Режимы передачи. Прототип поддерживает два режима:

- Открытый микрофон — непрерывная передача голоса. Пользователь нажимает «Включить микрофон», после чего все захваченные PCM-блоки автоматически отправляются на сервер и ретранслируются остальным участникам комнаты. При нажатии «Выключить микрофон» отправляется сигнал `MSG_VOICE_END`.

- PTT (Push-to-Talk) — передача только при удержании кнопки. При нажатии кнопки PTT отправляется `MSG_VOICE_START` и начинается захват; при отпускании — `MSG_VOICE_END` и захват останавливается. Режим удобен в условиях фонового шума и при участии в совещании нескольких человек из одного помещения.

Режимы взаимоисключающие: при включённом открытом микрофоне кнопка PTT игнорируется.

Формат аудио. Каждый голосовой блок — это 320 семплов PCM (16 бит, моно, 16 кГц), что соответствует 20 мс звука. Блок кодируется в Base64 и отправляется как JSON-сообщение типа `MSG_VOICE` с полями `room`, `sender`, `seq` (порядковый номер) и `data` (Base64-строка). Порядковый номер позволяет детектировать потерю блоков, хотя в текущей версии прототипа восстановление пропущенных блоков не реализовано.

Приём. На стороне получателя голосовые блоки декодируются из Base64, преобразуются в массив `numpy.int16` и подаются в кольцевой буфер воспроизведения. `sd.OutputStream` в режиме непрерывного callback забирает блоки из буфера и выводит на аудиоустройство. При переполнении буфера старые блоки отбрасываются, что предотвращает неограниченный рост задержки.

Управление приёмом осуществляется галочкой «Слышать собеседников». При снятии галочки аудиопоток останавливается (вызов `OutputStream.stop(`
но текстовые уведомления о начале/конце голосовой передачи продолжают отображаться в чате.

1.2.3. Передача файлов (изображений на проектор)

Третья функция прототипа — передача графических файлов (изображений) на клиент проектора для демонстрации на общем экране.

Сценарий. Пользователь нажимает кнопку «Изображение на проектор», выбирает файл через стандартный диалог ОС (поддерживаются форматы PNG, JPEG, BMP, GIF, WebP), после чего:

1. Клиент проверяет размер файла (лимит 12 МБ). При превышении выводится ошибка «Файл слишком большой».

2. Файл читается в бинарный буфер, кодируется в Base64 и отправляется как JSON-сообщение типа `MSG_IMAGE` с полями `room`, `sender`, `filename`, `data`.

3. Сервер рассылает сообщение всем пирам комнаты (роли `user` и `projector`). Таким образом, отправитель и другие пользователи также получают изображение — это служит подтверждением отправки.

4. Клиенты-проекторы при получении `MSG_IMAGE` декодируют Base64, открывают изображение через Pillow, масштабируют под размер канваса и отображают.

5. Клиент-отправитель создаёт уменьшенное превью (320×180) в нижней части окна и запись в чате.

Полноэкранный режим. Клиент проектора поддерживает переключение в полноэкранный режим по клавише `F11` и выход из него по `Escape`. В полноэкранном режиме скрываются панели управления и строка состояния, изображение занимает всё пространство экрана.

1.3. Запуск прототипа

Запуск прототипа ПК «CorpCom» выполняется в три этапа: запуск сервера, запуск клиентов и подключение к нужной комнате. Все компоненты запускаются из корневой директории проекта после однократного выполнения скрипта настройки `setup.sh`, создающего виртуальное окружение Python с необходимыми зависимостями.

1.3.1. Шаг 1. Сервер

Сервер (`corpcom.server`) запускается первым, так как клиенты и проектор подключаются к нему. На хосте, выделенном под сервер, выполняется:

```
$ cd <корень проекта>
$ ./run_server.sh
```

Сервер стартует на адресе `0.0.0.0` (все сетевые интерфейсы) и порту 8765 (значения по умолчанию). При успешном запуске в консоль выводится:

```
CorpCom server 0.0.0.0:8765 (token из CORPCOM_TOKEN)
```

Сервер работает в бесконечном цикле событий asyncio, ожидая входящие WebSocket-соединения. Для завершения используется `Ctrl+C`.

При необходимости изменить порт или токен аутентификации, они задаются через переменные окружения:

```
$ CORPCOM_PORT=9000 CORPCOM_TOKEN=my-safe-token ./run_server.sh
```

Аналогично можно указать конкретный IP-адрес для привязки (если хост имеет несколько интерфейсов):

```
$ CORPCOM_HOST=192.168.1.10 ./run_server.sh
```

1.3.2. Шаг 2. Клиент

На компьютере каждого пользователя запускается клиентское приложение с графическим интерфейсом:

```
$ cd <корень проекта>
$ ./run_client.sh
```

Открывается окно клиента CorpCom. В панели «Подключение (LAN)» заполняются поля:

- Сервер — IP-адрес хоста с запущенным сервером (по умолчанию `127.0.0.1` для локального тестирования);
- Порт — TCP-порт сервера (по умолчанию 8765);
- Имя — отображаемое имя пользователя в чате (до 64 символов);
- Комната — идентификатор комнаты. Все участники, указавшие одинаковую комнату, будут обмениваться сообщениями и голосом. По умолчанию `general`;
- Токен — секретный токен аутентификации. Должен совпадать с токеном, заданным при запуске сервера (переменная `CORPCOM_TOKEN`).

Нажатие кнопки «Подключить» инициирует WebSocket-соединение с сервером. Клиент отправляет AUTH-сообщение с токеном, ролью `user`, именем и комнатой. При успешной аутентификации:

- статусная строка меняется на «В сети: комната» с указанием списка участников;
- в чате появляется системное сообщение «Подключено к CorpCom»;
- кнопка подключения меняется на «Отключить».

При неверном токене сервер возвращает ошибку аутентификации, и клиент показывает диалоговое окно с текстом ошибки.

1.3.3. Шаг 3. Клиент проектора (опционально)

Если требуется демонстрация изображений на общем экране, на компьютере, подключённом к проектору или большому монитору, запускается:

```
$ cd <корень проекта>
$ ./run_projector.sh
```

В отличие от обычного клиента, проектор подключается с ролью `projector`. Поля «Сервер», «Порт», «Комната» и «Токен» заполняются аналогично клиенту. После нажатия кнопки «Старт» проектор переходит в режим ожидания изображений. При получении изображения оно автоматически отображается на всём канвасе. Клавиша `F11` переводит окно в полноэкранный режим.

1.3.4. Шаг 4. Подключение к нужной комнате

Механизм комнат реализован на стороне сервера: каждый пир при аутентификации указывает имя комнаты, и сервер маршрутизирует сообщения только между пирами одной комнаты. Таким образом, несколько рабочих групп могут одновременно использовать один сервер, не пересекаясь по данным.

Для подключения к нужной комнате достаточно указать её идентификатор в поле «Комната» клиента перед нажатием «Подключить». Идентификатор комнаты — произвольная строка до 64 символов. Рекомендуется использовать осмысленные имена, отражающие назначение комнаты: `general`, `lab`, `project-alpha`, `soveshchanie`.

Пример сценария. Три участника проектной группы хотят провести совещание с демонстрацией схем:

1. Администратор запускает сервер: `CORPCOM_TOKEN=meeting2026 ./run_server.sh`

2. Докладчик запускает клиент, указывает сервер, комнату `soveshchanie`, токен `meeting2026`, нажимает «Подключить».

3. Два других участника запускают клиенты с теми же параметрами.

4. На компьютере проектора запускается `./run_projector.sh` с комнатой `soveshchanie`.

5. Все видят друг друга в списке участников. Докладчик включает микрофон и начинает отправлять изображения слайдов на проектор. Остальные участники слушают доклад и задают вопросы текстом в чате.

После завершения совещания каждый участник нажимает «Отключить», проектор — «Стоп», администратор завершает сервер по `Ctrl+C`.